CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE

LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
RELATÓRIO: PRÁTICA 5

Belo Horizonte, 23 de Outubro de 2025

1.

```c
/*
* File:   ex01.c
* Author: Christopher Andrade
*
* Created on 16 de Outubro de 2025, 10:49
*/

#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

void main(void)
{
   INTCON2bits.RBPU = 0;   // habilita os resistores de pull-up internos
   ADCON1 = 0x0F;          // desliga entradas analógicas
   TRISB = 0xFF;           // configura PORTA B inteira como entrada
   TRISD = 0x00;           // configura PORTA D inteira como sáida

   while(1)
       LATD = PORTB;       //A cada ciclo copiar PORTB para LATD

   return;
}
```

Para esse programa, o valor de 1 byte armazenado no registrador PORTB é copiado para o registrador LATD a cada ciclo de execução resultando no efeito esperado de acionar cada um dos LEDs de acordo com a posição de suas respectivas chaves.

Além disso, antes de iniciar a parte cíclica do código algumas configurações de setup foram realizadas: primeiro habilitou-se os resistores de pull-up interno da PORTA B, essa decisão se deu por um aviso do fabricante do KIT XM118, o registrador ADCON1 com valor 0x0F configura todas as entradas como digitais, condizente com a natureza das variáveis desse exercício, por fim configurou-se todos os pinos da PORTA B como entrada e todos da PORTA D com saída.

2.

```c
/*
* File:   ex02.c
* Author: Christopher Andrade
*
* Created on 16 de Outubro de 2025, 10:49
*/

#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

void main(void)
{
   INTCON2bits.RBPU = 0;   // habilita os resistores de pull-up internos
   ADCON1 = 0x0F;          // desliga entradas analógicas
   TRISB = 0xFF;           // configura PORTA B inteira como entrada
   TRISD = 0x00;           // configura PORTA D inteira como sáida

   PORTD = 0xFF;           // Força todos os LEDs da PORTA D começarem apagados

   while(1)
       LATDbits.LATD7 = PORTBbits.RB0; // Copia bit 0 da PORTA B para bit 7 da PORTA D

   return;
}
```

As configurações de setup para esse programa são as mesmas do ex01.c, com exceção da atribuição do valor 0xFF no registrador PORTD, feita para forçar os LEDs a iniciarem apagados. Assim como no programa ex01.c a parte cíclica tem o mesmo funcionamento de copiar um valor do registrador PORTB para LATD, a diferença está no tamanho desse valor, aqui o valor é 1 bit, lá o valor é 1 byte.

3.

```c
/*
* File:   ex03.c
* Author: Christopher Andrade
*
* Created on 16 de Outubro de 2025, 10:49
*/

#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

void main(void)
{
   INTCON2bits.RBPU = 0;   // habilita os resistores de pull-up internos
   ADCON1 = 0x0F;          // desliga entradas analógicas
   TRISB = 0xFF;           // configura PORTA B inteira como entrada
   TRISD = 0x00;           // configura PORTA D inteira como sáida

   PORTD = 0xFF;           // Força todos os LEDs da PORTA D começarem apagados

   while(1)
   {
       // bit 7 define operação (0-AND 1-OR)
       // bit 0 e bit 2 são os operandos
       if(!PORTBbits.RB7)
           LATDbits.LD0 = (PORTBbits.RB0 && PORTBbits.RB1);
       else
           LATDbits.LD0 = (PORTBbits.RB0 || PORTBbits.RB1);
   }

   return;
}
```

As configurações de setup para esse programa são as mesmas do ex02.c. Na parte cíclica verifica-se o estado do MSB de PORTB, decidindo a operação lógica, sendo 0 para AND e 1 para OR, o resultado da operação entre os bits 0 e 1 de PORTB é atribuído ao LSB de LATD.

4.

```c
/*
* File:   ex04.c
* Author: Christopher Andrade
*
* Created on 16 de Outubro de 2025, 10:49
*/


#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

/* PROTÓTIPOS */
unsigned char count_bits();

void main(void)
{
   INTCON2bits.RBPU = 0;   // habilita os resistores de pull-up internos
   ADCON1 = 0x0F;          // desliga entradas analógicas
   TRISB = 0xFF;           // configura PORTA B inteira como entrada
   TRISD = 0x00;           // configura PORTA D inteira como sáida

   while(1)
       LATD = ~(count_bits()); //A cada ciclo LATD recebe um valor de 8 bytes [0 - 8]

   return
}

/*
   Descrição:  Conta o número de bits '1' no POTB
   Parâmetros: -
   Retorno:    Valor entre [0x00 e 0x08] em char (8 bytes) da contagem

*/
unsigned char count_bits(void)
{
   unsigned char num = 0x00;

   for(unsigned char mask = 0x01; mask != 0; mask = mask << 1)
       if((~PORTB & mask) != 0)
           num += 0x01;

   return num;
}
```

As configurações de setup para esse programa são as mesmas do ex01.c. Aqui o programa executa uma função que retorna um valor sem sinal de 256 bits [0 - 255] que representa a quantidade de bits 1 no PORTB, esse valor de retorno é atribuido ao LATD a cada ciclo de execução.

A ideia por trás da lógica da função é selecionar bit por bit do PORTB, utilizando uma máscara de bits que rotaciona para a esquerda a cada verificação, para a lógica foi utilizado um loop for que rotaciona a máscara, a cada ciclo se o bit selecionado for 1, incrementa-se uma variável de contagem ‘num’. Ao final retorna-se num com o valor de [0 - 8].

5.

```c
/*
* File:   ex05.c
* Author: Christopher Andrade
*
* Created on 16 de Outubro de 2025, 10:49
*/

#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

void main(void)
{
   unsigned char A, B;

   INTCON2bits.RBPU = 0;   // habilita os resistores de pull-up internos
   ADCON1 = 0x0F;          // desliga entradas analógicas
   TRISB = 0xFF;           // configura PORTA B inteira como entrada
   TRISD = 0x00;           // configura PORTA D inteira como sáida

   while(1)
   {
       A = (~PORTB & 0xF0) >> 4;   // A recebe os 4 bits MSB (Filtro e divisão por 16)
       B = (~PORTB & 0x0F);        // B recebe os 4 bits LSB (Filtro)
       LATD = ~(A + B);            // LATD recebe a cada ciclo A + B
   }

   return;
}
```

As configurações de setup para esse programa são as mesmas do ex01.c. Duas variáveis sem sinal de 256 bits foram declaradas (A e B), as quais armazenam os valores de 4 bits expressos na PORTA B. Para extrair o valor de A, seleciona-se, por meio de máscara de bits, os 4 MSBs e divide o valor por 16, equivalente a transladar os bits para direita 4 vezes, tornando A um valor [0 - 15] correspondente a parte alta de PORTB. Para B, basta selecionar os 4 LSBs de PORTB, resultando em um valor [0 - 15]. Por fim, basta atribuir a LATD a soma de A e B.